

HELIOS

Project report: an LLM inference engine, what it does, and what it does not

Repository	github.com/anish41338/helios
Commit	06a9055
Report generated	28 July 2026
Language	Python 3.12 (no Rust, no C++, no CUDA C)
Source lines	8,242 in python/, 3,404 in tests/, 3,902 in bench/
Tests	269 run on CPU, plus 9 requiring an NVIDIA GPU
Simulation seeds run	50,000 (scheduler), 5,000 (with the transfer FSM)
Hardware used	Windows CPU (torch 2.5.1+cpu); Tesla T4 via Kaggle for GPU gates
Commits	23

This document records what was built and measured. It does not propose further work, and it does not present a limitation as an opportunity. Numbers that were measured are labelled measured; numbers that come out of an arithmetic model are labelled modelled, on the same line. Sections 11, 12 and 13 list what is absent, what was got wrong, and what only showed up on real hardware.

1	What the software is, and the module inventory
2	The paged KV allocator
3	The scheduler
4	Deterministic simulation testing, and the fifteen bugs it found
5	The executor: model, batching, Triton kernel
6	Quantization: INT4 weights, AWQ, INT8 KV cache
7	Quantization-asymmetric speculative decoding, and the acceptance rate
8	API, streaming and metrics
9	The KV transfer state machine (not called by the engine)
10	Verification: tests, exact-equality properties, mutation testing
11	What is not built
12	Errors made during the build
13	What running on a GPU exposed
14	Reproduction and provenance

1 What the software is

An LLM inference server. It loads a Llama-family or Qwen2 checkpoint in safetensors format, holds attention state in a paged KV cache it manages itself, schedules many concurrent requests through a continuous-batching loop, and serves them over an OpenAI-compatible HTTP API with incremental streaming.

The transformer forward pass is written from scratch — grouped-query attention, rotary position embeddings, SwiGLU feed-forward, RMS normalisation — rather than calling into transformers. The reason is not self-sufficiency for its own sake: attention has to read and write physical blocks chosen at runtime by this project's allocator, and no stock implementation exposes that.

It runs real pretrained weights. It has been driven end to end on Qwen2.5-0.5B on both a CPU and a Tesla T4, producing coherent text through the full HTTP path. It is not a wrapper, and the generation loop, KV management, batching, sampling and API surface are all this project's own code.

It is not production infrastructure. There is no multi-node support, no authentication, no rate limiting beyond the admission buckets, no model hot-swapping, no tensor or pipeline parallelism, and no persistence. It runs in a single process.

Structure

The code splits at one boundary. The **core** — allocator, scheduler, prefix cache — contains no clock, no I/O, no threads, no unseeded randomness, and no iteration over unordered containers. The **executor** contains all the PyTorch. They exchange only ExecStep and ExecOutputs values.

That boundary exists so a fabricated executor can be substituted for the real one, which is what makes the scheduler testable under fault injection. Everything in section 4 depends on it.

Module inventory

Module	Lines	Contents
core/scheduler.py	1206	Continuous batching, admission, preemption, chunked prefill, speculation gating
core/vopr.py	857	Deterministic simulation harness: workload generation, fault injection, invariant checks, replay
core/allocator.py	602	Paged block allocator: block tables, ref counts, copy-on-write, host swap tier, invariants I1-I7
core/disagg.py	543	KV transfer state machine (not called by the engine -- see section 9)
core/prefix_cache.py	307	Radix trie over block-aligned prefixes, LRU eviction, pinning
core/types.py	211	Requests, sequences, sampling params, SLO classes
core/execstep.py	167	The scheduler/executor value contract
exec/quant.py	602	INT4 packing, group-wise quantization, AWQ scale search, QuantLinear
exec/model.py	524	Llama/Qwen2 forward pass: GQA, RoPE, SwiGLU, RMSNorm, batched and single-sequence paths
exec/runner.py	433	ExecStep execution, batched prefill/decode, speculative decode
exec/paged_attn.py	408	Paged KV storage and attention; INT8 quantized cache variant
exec/qassd.py	376	Dual-precision model, acceptance measurement, speedup model
exec/loader.py	301	safetensors loading, toy-model generation
exec/triton_attn.py	183	Fused paged decode attention kernel
exec/sampler.py	96	Greedy, temperature, top-k, top-p, per-request seeds

engine.py	559	Scheduler + executor + tokenizer + metrics + streaming
api/server.py	548	OpenAI-compatible HTTP, SSE streaming, Prometheus
cli.py	309	serve / generate / vopr / info / make-toy-model

Total: 8,242 lines under python/. Test code adds 3,404 lines and the benchmark and reporting harness 3,902.

2 The paged KV allocator

A conventional implementation reserves a contiguous KV buffer per sequence sized to the maximum context length. A 100-token request occupying a 4096-token reservation wastes 97.5% of it. This allocator instead hands out fixed-size blocks and maps them per sequence through a block table, so waste is bounded by at most one partially-filled block per sequence.

What it implements

- **Block tables** mapping logical block index to physical block id, with a `filled_in_last` count so a partially used tail block is tracked exactly.
- **Reference counting** per physical block, with references coming from two distinct sources: live block tables and prefix-cache retention. They are counted separately so that an accounting error in one is attributable.
- **Copy-on-write.** Two sequences may share physical blocks after a fork or a prefix-cache hit. A write to a shared block copies it first. The case that matters is a shared block that is only partially filled, because it has spare room and therefore looks free to write into.
- **A host swap tier.** Blocks can be moved to a second, larger, slower pool instead of being discarded, and swapped back later.
- **A watermark** reserving a fraction of the pool, so the engine cannot reach a state where every running sequence needs one more block and none exists.

The seven invariants

Asserted after every allocator operation in tests, and after every scheduler step in simulation:

I1	A block is in exactly one free list, or referenced, never both
I2	<code>ref_count</code> equals block-table references plus prefix-cache references
I3	<code>free + referenced == total</code> , per tier
I4	No holes in any block table
I5	<code>filled_in_last</code> is in <code>[1, BLOCK_SIZE]</code> for a non-empty sequence
I6	A shared block is never written in place
I7	Committed blocks stay under the watermark

I7 is the one that caught a real bug. `blocks_needed_to_append` returned zero blocks for a shared, partially-filled tail on the grounds that it had room. It did, but writing to it required copying it first, so the allocation overshot the reserve. That is bug 003 in section 4.

Sizing

Block count is derived from a byte budget rather than configured: $2 \times n_layers \times block_size \times n_kv_heads \times (head_dim \times dtype_bytes + scale_bytes_per_token)$. The trailing term is zero for an unquantized cache and 2 for the INT8 cache, which stores one fp16 scale per token per head. It is inside the formula rather than applied afterwards because the engine divides its budget by this number to get a block count, and an understated divisor hands out more blocks than the cache physically has.

3 The scheduler

1,206 lines, and the largest single component. It decides, every iteration, which sequences run and what work each contributes.

Continuous batching

Scheduling decisions are made per forward pass, not per batch. A sequence that emits an end-of-sequence token leaves immediately and a waiting sequence takes its place in the same iteration. `_reap_finished` runs before `_admit` specifically so the freed blocks are available in that step rather than the next one.

Chunked prefill

A long prompt is split across steps against a token budget (`max_num_batched_tokens`), so a single large prefill cannot monopolise an iteration and inflate inter-token latency for every sequence already decoding. Successive chunks write into the same block list at increasing offsets and each attends over everything cached so far, which is what makes chunked output identical to single-shot. The step builder also reserves one token of budget per decoding sequence before packing prefills, so prefill cannot starve decode.

Admission and starvation

Requests are ordered by (SLO class, arrival step, sequence id) and admitted against per-class token buckets refilled by *step count*, never by elapsed time. A request that does not fit is skipped rather than blocking the queue — but after 32 consecutive skips the queue head receives an exclusive reservation. Those two rules exist because the simulation harness found the opposite livelocks separately: stopping at the first request that does not fit starves small requests (bug 006), and always skipping starves large ones (bug 014).

Preemption

Under memory pressure a victim's blocks are reclaimed either by discarding its KV and recomputing later, or by swapping to the host tier. The choice is a ratio test: swap only when generated length exceeds four times prompt length, because below that recomputing is cheaper than moving the bytes twice. The last running sequence is never preempted — there is no other work for the freed memory to serve, so it is pure loss (bug 009).

Prefix cache

A radix trie over block-aligned prompt prefixes. A matching prefix lets a new request adopt existing physical blocks instead of recomputing them. Eviction is LRU, and blocks in use are pinned. Matching is block-aligned because a partial block cannot be shared without copy-on-write, which would negate the saving.

Measured on a workload with a long shared prefix: **1.49×** throughput against the same workload with the cache disabled (218.6 against 146.3 output tokens per second), and time-to-first-token from 828 ms to 536 ms. On the default workload, whose shared prefix is 32 tokens, the cache shows no benefit; that is not a defect but it does mean the headline figure is regime-specific.

Determinism

The scheduler makes no random choices, reads no clock, and iterates no unordered container. A test asserts the module imports neither `time` nor `random`. This is a hard constraint rather than a style preference: a single wall-clock read would make a failing simulation seed unreplayable, and section 4 would not work.

4 Deterministic simulation testing

The scheduler and allocator are driven single-threaded by a harness in which every source of nondeterminism is derived from one integer seed: the workload, arrival burstiness, configuration values, cancellations, and the fault schedule. The clock is an integer counter. The executor is fabricated. Invariants are asserted after every step.

What a seed randomises

- Configuration: block size from {8, 16, 32}, pool size 24-200 blocks, host tier 0-64 blocks, max sequences 2-32, token budget from {128, 256, 512, 2048}, context limit from {256, 512, 1024}, watermark from {0, 0.01, 0.05}, and whether chunked prefill, the prefix cache, speculation and swap are enabled.
- Workload: 1-60 requests, Poisson-ish arrivals with occasional ten-fold bursts, prompt and output lengths, SLO class, and whether a canned shared prefix is used so the prefix cache actually hits.
- Faults: 0-6 injected executor failures drawn from OOM, CUDA error, timeout, transfer stall, transfer checksum, transfer partial, and swap-tier-full, at randomly chosen steps; plus forced acceptance rates of 0% and 100% as adversarial extremes.
- Cancellations at arbitrary later steps.

What is asserted

The seven allocator invariants, plus structural properties added as bugs demanded them: no sequence in two queues; a waiting sequence owns no KV and no cache pin; a finished sequence holds neither; every sequence is reachable from a queue or terminal and reaped; cache pins are attributable to a live sequence. Then two global properties at the end of each run: **liveness** (every submitted request reaches a terminal state) and **no leaks** (every block with a positive reference count has an owner).

Result

```
50,000 seeds - 0 failures (3,807 s single-threaded)
20,000 seeds - 0 failures (re-verified after the executor was rewritten)
5,000 seeds - 0 failures (re-verified after the transfer FSM was added)
```

Any failure replays exactly from its seed: `python -m helios.cli vopr --seed 918273 --replay`.

The fifteen bugs it found

All fifteen were found by the harness rather than by hand, and none were seeded deliberately. Each is fixed, each fix carries a regression test, and each is named in a comment at the fix site. **Thirteen of the fifteen were liveness failures** — livelocks and starvation — rather than data corruption. In those cases the engine kept running and reported healthy metrics while completing nothing.

#	Symptom	Root cause
001	Deadlock, 0/44 requests finished	With chunked prefill off, a prompt longer than the per-step token budget was unschedulable, and admission broke on it, blocking every smaller request behind it
002	Same deadlock, remaining case	A prompt within <code>max_model_len</code> but larger than the whole KV pool was accepted and queued forever; <code>max_model_len</code> was never validated against KV capacity
003	17 violated: committed 63 > usable 62	<code>blocks_needed_to_append</code> returned 0 for a shared, partially-filled tail block. It had spare room, but the write still required a copy-on-write, so allocation overshot the watermark
004	Livelock, cached blocks never reclaimed	The prefix cache held blocks from finished sequences and nothing evicted them under memory pressure
005	Livelock, eviction found no victims	prefix-cache acquire/release were unbalanced: a preempted and re-prefilled sequence acquired twice and released once, leaving holders > 0 forever

006	Head-of-line livelock	Admission stopped at the first request that did not fit, starving smaller requests behind it when no running sequence remained to free memory
007	Cache nodes permanently pinned	A sequence that attached a cached prefix then failed allocation returned to the waiting queue still holding the pin, making the memory it needed unevictable
008	Starvation with memory available	No backstop when progress stopped. The progress metric also counted prefill tokens, which a repeatedly-preempted sequence inflates indefinitely
009	271 preemptions of a single sequence	Preempting the last running sequence destroys progress for no benefit -- there is no other work to free memory for
010	68 OOM-preempts, each redoing a 204-token prefill	Admission allocated for the prompt but reserved nothing for the tokens the sequence would generate
011	Most of the pool cached and unevictable	Publishing a prompt to the cache also pinned it for the publisher's lifetime
012	Two sequences recompute-preempting each other	The growth-headroom check tested total capacity rather than currently free blocks, so both were admitted believing they had room
013	Entire KV pool leaked	A sequence that finished outside the running list became an orphan: marked finished, in no queue, holding 32/32 blocks. <code>_reap_finished</code> only walked running
014	Starvation with 9 free blocks	Requests needing 3 blocks were refused while a 14-block request sat ahead of them and the running set had stabilised
015	Sequence admitted and re-prefilled every step forever	A sequence needing the whole pool skipped the growth guard once it already held a block table mid-prefill

Two of the fixes were themselves wrong and the harness said so within seconds. The fix for bug 004 dropped the pass rate from 193/200 to 157/200 by exposing bug 005 underneath it. The fix for bug 011 surfaced bug 013.

A coverage gap, and what it showed

Copy-on-write turned out to be unreachable in simulation: 0 of 300 seeds exercised it, because the prefix cache shares only whole blocks, so no sequence ever wrote into a shared partial tail — the exact case bug 003 lived in. Adding a fork operation to the harness produced full coverage of that path. **Reintroducing bug 003 then still passed 300 of 300 seeds.** The line was executed; the conditions that make it fail were not reached. Making the forks persist across steps did catch it, at the cost of roughly sixty spurious liveness failures per six hundred seeds, because the scheduler cannot see those shadow tables and so can neither evict nor preempt them. That change was reverted and the case is pinned by a targeted test with exact control of the pool state instead.

5 The executor

The model

A from-scratch implementation of the Llama 3.x / Qwen2.5 architecture: grouped-query attention, rotary embeddings using the rotate-half convention, SwiGLU feed-forward, and RMS normalisation computed in fp32 regardless of parameter dtype. Weights load from safetensors, read lazily per tensor so peak host memory stays near one tensor rather than the whole model.

The loader raises rather than skipping any checkpoint tensor it has no parameter for, with an explicit allowlist for genuine buffers. That is a response to a specific failure: Qwen2 places biases on the query, key and value projections and Llama does not; the model hardcoded no bias and the loader silently discarded three tensors per layer. The model loaded successfully and generated fluent nonsense.

Batched execution

All decoding sequences in a step run in one forward pass. Sequences are concatenated along a single token dimension, so every projection and the MLP become one large matrix multiply; attention remains per sequence over its own paged KV, gathered into a right-padded tensor and masked. Prefill chunks are concatenated the same way, and the language-model head is applied only at the rows that will be sampled — with a 128k vocabulary, projecting every prompt token would cost more than the rest of the layer stack.

Configuration	Output tok/s	vs full engine
helios_full (all mechanisms)	409.4	—
baseline_unbatched_executor (one pass per sequence)	183.1	2.24× slower
baseline_static_batch_8	361.2	1.13× slower
baseline_hf_loop (no engine, no KV reuse)	71.6	5.72× slower

These are CPU measurements on a randomly-initialised toy model. The absolute tokens-per-second figures are meaningless; the ratios between configurations are the result. The same ablations on a Tesla T4 gave 3.14× for decode batching and 1.50× against static batching.

The Triton kernel

Paging costs a gather: a sequence's KV is scattered, so the PyTorch path collects it into a contiguous tensor before computing scores. The fused kernel instead runs one program per (sequence, query head), walks the block table, and streams each block through an online-softmax accumulator, so neither the gathered KV nor the score vector is written to memory. State per program is $O(\text{head_dim})$ rather than $O(\text{context_len})$.

Measured on a Tesla T4 (sm_75, CUDA 12.8, Triton 3.6.0) at 32 sequences by 512 context: **14.089 ms to 2.676 ms, a 5.26× speedup**, with 9 of 9 parity tests passing against the PyTorch paged path at atol = rtol = $2e-2$. Three independent runs gave 5.26×, 5.11× and 5.58×. The kernel was written on a machine with no NVIDIA GPU and marked unverified until a device ran the gate.

6 Quantization

INT4 weights

Weights are quantized to 4 bits with asymmetric group-wise scaling: two nibbles packed per byte, one fp16 scale and one uint8 zero-point per group of 128 input channels per output channel. Asymmetric rather than symmetric because at 4 bits there are only 16 levels and per-group weight distributions are not centred. uint8 packing rather than the int32 layout AWQ ships, because packing eight nibbles into a signed int32 puts a value in the sign bit and relies on wrap-around semantics PyTorch does not promise.

Quantization is applied in-process after loading an fp checkpoint, not read from a pre-quantized artifact. Loading an existing AWQ or GPTQ checkpoint is not implemented.

Measurement	Value
Compression vs fp16, single layer	3.82× (measured, after use)
Compression vs fp32, Qwen2.5-0.5B, 168 layers	7.38× on quantized layers
Whole-model compression, Qwen2.5-0.5B	1.96× (lm_head stays full precision)
Round-trip error bound	$ \text{error} \leq \text{scale}/2$, asserted elementwise
Speed on CPU	SLOWER than fp32 — see below

There is no int4 matrix-multiply kernel. QuantLinear dequantizes to floating point and then calls a normal GEMM, so on a CPU it performs strictly more work than the fp32 layer it replaces and runs slower. The memory reduction is real and measured; the speed benefit that W4A16 exists for is not implemented and not claimed. Every speedup figure involving quantization in this project is labelled modelled.

AWQ activation-aware scaling

Quantization error on an input channel is weighted by that channel's activation magnitude when it reaches the output. Scaling a salient channel's weights up before quantizing gives it more of the 16 available levels, and a matching divide on the activation cancels it exactly: $(x/s) @ (w \cdot s) = x @ w$ for any positive s , so scaling is free in exact arithmetic and only moves where the error lands. Implemented as a grid search over the exponent in $s = \text{activation_absmean} ** \alpha$ for α in $[0, 1]$, minimising output error on captured calibration activations.

Activations are captured with forward hooks over calibration prompts, subsampled by a deterministic stride so calibration is reproducible. Measured across 168 layers of Qwen2.5-0.5B: mean relative per-layer output error **0.01167 with round-to-nearest against 0.00739 with AWQ, a 1.58× reduction**. Section 7 records that this reduction does not translate into better draft acceptance.

INT8 KV cache

The paged KV cache has an INT8 variant with symmetric per-token, per-head scales stored in fp16. Per-token rather than per-tensor because attention is a dot product over head_dim, so a single outlier token would otherwise set the scale for every token sharing it. Symmetric rather than asymmetric because a zero point would have to be carried through the score matmul as a correction term rather than folding into one multiply. It is a drop-in subclass: writes quantize, reads dequantize, and no attention code changes.

The round-trip error bound is $|\text{error}| \leq s/2 + |q| \cdot |s - \text{fp16}(s)|$. The second term is easy to omit and is what makes an $s/2$ -only bound fail: the scale is stored in fp16, so reconstruction multiplies by a slightly different scale than the one used to divide. At $|q| = 127$ it reaches about 6% of s .

256 KiB KV pool, 16 requests	fp32 KV	INT8 KV	Change
Mean resident decode batch	4.47	10.31	2.31× more

TTFT p50	2201 ms	1356 ms	1.62× better
Output tok/s	73.8	54.0	27% worse

The memory mechanism works and the throughput does not follow. Halving the cache admits more sequences and improves time-to-first-token, and the dequantization cost on a CPU exceeds what the extra concurrency returns. Whether it pays on hardware where reading the KV cache is the decode bottleneck was not measured.

The pool size matters and the first attempt got it wrong. Run in a 4 MiB pool, both configurations kept every sequence resident (mean batch 14.95 in both), so the comparison measured only dequantization overhead and INT8 appeared 20% slower with no offsetting benefit available to it.

7 Quantization-asymmetric speculative decoding

Speculative decoding drafts several tokens cheaply and verifies them in one parallel pass, committing the longest prefix the verifier agrees with. The variant implemented here drafts from a 4-bit quantization of the target's own weights rather than from a separate small model.

Mechanism

- Draft gamma tokens sequentially with the int4 model, writing into its own quantized KV cache.
- Verify all gamma positions in one full-precision forward pass.
- Accept the longest prefix where the draft matches the verifier's argmax.
- Emit one bonus token from the verifier at the first mismatch, so a rejection never stalls the sequence.
- Re-synchronise the draft's KV over the committed span. The draft wrote keys for every token it proposed; past the first rejection those describe a continuation that was discarded, and paged storage has no notion of truncation.

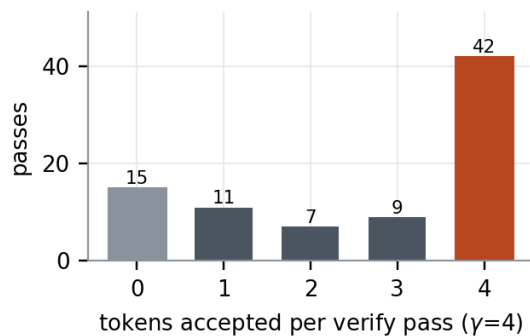
Output is unchanged by speculation. An accepted token is one the target's argmax agreed with, and a rejection emits the target's argmax, so the committed sequence is the target's greedy sequence whatever the draft proposes. A test runs identical prompts with and without the int4 draft and requires identical token ids.

The acceptance measurement

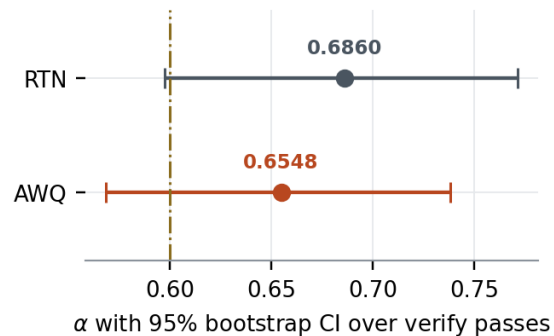
The design depends on how often the degraded draft agrees with the verifier. A threshold of 0.6 was fixed in advance, in the design document, before any of this was built.

Quantity	Value
Model	Qwen2.5-0.5B, fp32 target, int4 AWQ draft, group size 128
Prompts	8, spanning factual recall, arithmetic, code and prose
Calibration	4 prompts, disjoint from the evaluation set
Tokens drafted	336
Tokens accepted	220
Acceptance rate (alpha)	0.6548 (measured)
Pre-committed kill threshold	0.60 — passed
Design target	0.78 — not met
Measured tokens per verify pass	3.12
Accepted-run histogram (gamma=4)	{'0': 15, '1': 11, '2': 7, '3': 9, '4': 42}
Optimal gamma from the model	2 (planned: 4)
Modelled speedup at that gamma	1.39× (modelled, not measured)
Modelled speedup at gamma=8	0.94× — a net loss (modelled)

Measuring this on a CPU is legitimate because alpha is the probability that two weight matrices produce the same argmax on the same context. It is arithmetic and does not depend on how fast either forward pass runs. The speedup does depend on that, is not implemented, and is labelled modelled everywhere it appears.



Accepted-run lengths per verify pass.



AWQ against RTN, 95% bootstrap CIs.

The accepted-run distribution is bimodal: 42 of 84 verify passes accepted the entire draft and 15 accepted none. Acceptances are therefore correlated, which contradicts the independence assumption in the standard speedup formula. The measured tokens per verify pass is 3.12 against 2.55 predicted under independence, so the formula understates the value by 22% on this data.

AWQ did not improve acceptance

AWQ reduces mean per-layer output error by 1.58 $\times$. Acceptance with AWQ is **0.6548** and with plain round-to-nearest **0.6860** — nominally worse. Bootstrapping over verify passes rather than tokens (because acceptances are correlated, so treating 336 tokens as independent trials would understate the variance) gives 95% intervals of [0.568, 0.738] and [0.598, 0.771], and a permutation test gives **p = 0.64**. The two are indistinguishable at this sample size.

No conclusion is drawn beyond that. A possible explanation is that AWQ minimises squared output error while acceptance depends only on whether the argmax survives, which is a coarser property — but that was not tested and is not a result.

Measured cost when speculation actually runs

Batch capped to 8, 16 requests	Output tok/s	Against no speculation
Speculation off	200.4	—
Symmetric, gamma=2	62.9	3.19 $\times$ slower
Symmetric, gamma=4	65.7	3.05 $\times$ slower
QASSD, gamma=2	141.7	1.41 $\times$ slower

Speculation as implemented gives up batched decode. The draft loop is serial and runs per sequence, so with 8 resident sequences a speculative step costs 8 $\times$ (gamma+1) unbatched forward passes where an ordinary step costs one batched pass. On this CPU that trade is never worth making, and no configuration of speculation is faster than not speculating.

QASSD costs less than symmetric speculation here only because the adaptive gate disabled it. On the toy model the int4 draft agrees 12.5% of the time; measured, the gate turned speculation off after a single step and 1 of 47 decode steps speculated. The gate reacts to batch size above 8 and to measured acceptance below 0.5 over a sliding window.

Memory cost

Cost	Measured
Draft weights (int4 copy, both precisions resident)	+41% on weights
Draft KV shadow (a second, quantized KV cache)	+28% of the main cache

Combined, as reported by `engine.memory_report()`

1.28× total resident

Enabling this feature increases memory. It does not reduce it. A configuration that keeps only the int4 weights does reduce memory, but contains no speculation.

8 API, streaming and metrics

An OpenAI-compatible HTTP surface: `/v1/completions`, `/v1/chat/completions`, `/v1/models`, `/health` and `/metrics`, plus a namespaced `helios` block for SLO class, speculation depth and prefix-cache control that standard clients ignore.

Concurrency

The engine is single-threaded by design, so all stepping happens under one asyncio lock. Rather than a background task, whichever request is waiting drives the engine forward and distributes results to every waiter. That inversion is necessary: a perpetual loop starves the handlers it serves under a cooperative event loop, because stepping is CPU-bound and never truly awaits.

Streaming

Server-sent events, one frame per text delta. Deltas are produced by decoding the whole output each step and taking the new suffix, rather than decoding tokens individually: a BPE token is not a character, and a multi-byte codepoint spans several tokens, so per-token decoding would put invalid UTF-8 on the wire. Speculative decoding makes frame sizes uneven because a verify pass can commit several tokens at once.

Verified against Qwen2.5-0.5B over the real HTTP path: a 12-token completion produced **13 SSE frames concatenating byte-identically** to the non-streamed response for the same prompt. A test asserts that equality, and a second asserts the per-request delta queues are deregistered afterwards.

```
POST /v1/completions {"prompt": "The capital of France is",
                     "max_tokens": 12, "temperature": 0.0}
-> " Paris. It is the largest city in Europe and the second"
    usage: prompt_tokens 5, completion_tokens 12, cached_tokens 0

POST /v1/completions same body plus "stream": true
-> 13 data frames, then a terminal frame carrying finish_reason,
    then data: [DONE]
-> concatenated text identical to the non-streamed response
```

Metrics

A Prometheus text endpoint exposing 18 gauges and counters: queue depths, KV block usage and utilisation, prefill and decode token totals, finished and aborted counts, preemptions by kind, prefix-cache hit ratio, executor faults, step counts, and, when a quantized draft is configured, the measured acceptance rate and weight overhead ratio. Latency summaries (TTFT and TPOT percentiles, and goodput against per-class SLO targets) are computed from completed requests.

9 The KV transfer state machine

This component is not called by the engine. The only non-test module that imports it is the simulation harness. It is a verified design, not a feature that can be enabled, and it is described here on that basis.

It models migrating a prompt's KV cache from a prefill partition to a decode partition. Seven states with an explicit legal-transition table: pending, reserved, sending, sent, received, committed, and the terminal failed and aborted. An unlisted transition raises.

The design separates sent (the sender believes it is done) from received (the receiver confirms). Five fault kinds are modelled — receiver out of memory, link timeout, checksum mismatch, lost acknowledgement, and mid-flight abort — and each returns block lists for both partitions, because a failure leaves blocks reserved on the receiver that the sender does not know about and blocks pinned on the sender that the receiver cannot see. Retries are bounded and fall back to re-prefilling.

Half of all simulation seeds drive it alongside the scheduler under randomised faults, asserting six state-machine invariants and two pool invariants the state machine cannot check itself. Coverage measured across 200 seeds reached every fault kind: abort in 80 seeds, link timeout in 41, checksum mismatch in 30, receiver OOM in 22, with retry in 72 and re-prefill fallback in 56.

Detection was verified rather than inferred from coverage. Reintroducing the partial-failure leak (returning the sender's blocks but not the receiver's) was caught by **31 of 60 seeds**. Permitting an abort after the bytes are already sent was caught by **15 of 60**.

It also computes whether transferring a prompt's KV is cheaper than recomputing it, from bandwidth rather than assumption. For a 2000-token prompt on a 32-layer model with 8 KV heads at head_dim 128 in fp16, the KV is 262 MB: 0.9 ms over NVLink at 300 GB/s, 21.8 ms over PCIe 3.0 at 12 GB/s, and 209.7 ms over 10 GbE, against 250 ms to re-prefill at 8000 tokens per second.

10 Verification

269 tests run on a CPU and a further 9 require an NVIDIA GPU, for 278 in total. Distribution by area:

Test file	Tests	Lines
tests/allocator/test_allocator.py	57	308
tests/parity/test_parity.py	38	684
tests/allocator/test_prefix_cache.py	36	229
tests/e2e/test_api.py	27	292
tests/scheduler/test_scheduler.py	26	423
tests/quant/test_quant.py	26	368
tests/disagg/test_disagg.py	26	362
tests/quant/test_qassd.py	23	308
tests/quant/test_kv_quant.py	10	205
tests/parity/test_triton_parity.py	9	174

Properties asserted by exact equality

Property	Guarantee
Chunked prefill against single-shot prefill	Bit-identical
Speculative against non-speculative decode	Bit-identical, gamma in {1,2,4,8}
QASSD against unspculated decode	Identical token ids
Batched against sequential decode	Identical logits, order-invariant
Prefix cache on against off	Identical output
Engine against a naive generation loop	Identical tokens
Output across 14 forced recompute preemptions	Identical
Streamed deltas against the non-streamed response	Byte-identical
INT4 pack/unpack	Exactly invertible
Paged attention against a dense reference	Within float precision
Triton kernel against the PyTorch paged path	Within fp16 tolerance on a T4

Mutation testing

Three deliberate bugs were introduced to confirm the tests that exist to catch them actually do:

Injected bug	Caught by
Commit the draft's token instead of the verifier's at a rejection	test_asymmetric_speculation_matches_the_target_exactly
Quantized copy_block drops the per-token scales	test_copy_block_moves_the_scales
Block sizing formula ignores the KV scale bytes	test_allocator_sizing_accounts_for_the_scales

The first result is worth stating precisely. When the accept logic was mutated, **every symmetric-speculation parity test passed**, because at an acceptance rate of 1.0 the draft token and the verified token are the same and the bug is invisible. Only the asymmetric test failed.

End-to-end verification on real weights

Qwen2.5-0.5B (630M parameters as loaded, 494M plus an untied output projection) was run through the engine on two devices. On a Tesla T4 in fp16 and on this CPU in fp32, the same prompts produced the same continuations:

```
"The capital of France is" -> " Paris. It is the largest city in Europe
                             and the second largest in the world..."

"Water boils at"           -> " 212 F and ice melts at 32 F. What is the
                             number of degrees..."

"def add(a, b):"           -> "\n    return a + b\n\ndef subtract(a, b):\n
                             return a - b\n\ndef multiply(a,"
```

Paging, block tables, rotary embeddings, GQA head expansion, normalisation, batched prefill, batched decode, the dtype handling and sampling must all be simultaneously correct for a pretrained model to produce this. The toy model used elsewhere in the test suite has random weights and is written by the same code that reads it, so it can only demonstrate self-consistency.

The content is the model's own knowledge and not a claim about the engine. Paris is not the largest city in Europe.

11 What is not built

Absent	Detail
Fused int4 GEMM	QuantLinear dequantizes and calls a normal GEMM. On CPU it is slower than fp32. The memory reduction is measured; the speed benefit W4A16 exists for is not implemented.
vLLM comparison	bench/vllm_baseline.py is written with matched controls — same model directory, dtype, KV byte budget, seeded arrival trace, discarded warm-up — and has never been executed. vLLM requires CUDA.
W4A4 (4-bit activations)	Not implemented. The available GPU has no fast int4 activation path.
Multi-device disaggregation	The transfer state machine exists and is verified. No transport, no second device, and the engine does not call it.
Rust frontend or scheduler	Not written. Everything is Python.
gRPC, shared-memory IPC	Not written. Single process.
Tensor or pipeline parallelism	Not written. Single device.
Pre-quantized checkpoint loading	AWQ and GPTQ artifacts cannot be read. Quantization is applied in-process to an fp checkpoint.
Acceptance measurement above 0.5B parameters	Not measured. Whether alpha improves with model size was not tested.
GPU measurement of the INT8 KV cache	The concurrency benefit was measured on CPU; whether it pays where bandwidth is the bottleneck was not measured.
Beam search, n>1 sampling, logit bias, logprobs	Not implemented. The API accepts logprobs and returns null.
Authentication, rate limiting, multi-tenancy, persistence	Not implemented.

One caveat applies to every CPU performance number in this document: the benchmark model is a small randomly-initialised toy, so absolute tokens-per-second figures are meaningless. Only the ratios between configurations are results. The GPU figures come from a Tesla T4 on Kaggle and are recorded in artifacts.

12 Errors made during the build

Listed because they are part of the record and several changed the conclusions.

Error	How it was found and what changed
A memory optimisation that was a memory regression. QuantLinear cached the dequantized fp weight, so after one forward pass an int4 layer held 10.6 MiB against 8.4 MiB for the fp16 layer it replaced — a 1.26x increase, reported as a 3.82x saving because stored_bytes() counted only the packed form	Found by an audit run after the code shipped green. Every quantization memory claim in the project was false in practice. Caching is now opt-in and off by default; resident_bytes() was added.
A benchmark that measured nothing. The speculation ablation ran at the default batch of 12-16, above the gate's threshold of 8, so 0 of 23 decode steps speculated	Found by instrumenting the gate. The row labelled speculation was reporting scheduling variance. A separate suite now caps the batch.
An incomplete memory figure. memory_overhead() covered weights and omitted the second KV cache QASSD allocates	Found in the same audit. The number was true and incomplete. engine.memory_report() now returns both.

An overstated claim. The transfer state machine was listed as built alongside paged attention, when nothing outside the test harness imports it	Found by grepping for its consumers. Relabelled.
A wrong conclusion published as a finding. "Continuous batching needs a GPU to pay" was documented, because the measured batching win was zero	A matmul microbenchmark showed roughly 10x from batching GEMMs on CPU at 32 sequences. The engine was running one forward pass per sequence — a missing optimisation, not a hardware property. Fixing it gave 2.24x.
An arithmetic error in a comment that inverted a verdict. A 2000-token KV cache was stated as 2.6 GB; it is 262 MB	Found by a test written against the real formula. The tenfold error changed the PCIe verdict from "transfer wins by 11x" to "roughly break-even".
Three mechanisms benchmarked outside the regime they exist for: the prefix cache on a 32-token shared prefix, prefill batching behind long generations, and the INT8 KV cache in a pool that did not bind	Each appeared worthless or harmful until the workload was constructed so the mechanism had something to do. Dedicated suites now exist.
A benchmark contaminated by a concurrent job on the same machine, inverting the gamma=2 / gamma=4 ordering	Re-run on an idle machine; the inversion disappeared.
Two bug fixes that were themselves wrong	The fix for scheduler bug 004 dropped the simulation pass rate from 193/200 to 157/200 by exposing bug 005. The fix for bug 011 surfaced bug 013.
A test-coverage gap mistaken for coverage. Adding forks to the harness gave full line coverage of copy-on-write, but reintroducing the original bug still passed 300 of 300 seeds	Coverage of a path is not detection of its failure conditions. The case is pinned by a targeted test instead.

13 What running on a GPU exposed

Four bugs were found by putting the code on a Tesla T4 while 178 tests were passing. Three of them could not have been caught on the development machine.

Bug	Why it was invisible on CPU
Qwen2 attention biases silently dropped. Qwen2 places biases on q/k/v projections; Llama does not. The model hardcoded bias=False and the loader skipped checkpoint tensors with no matching parameter	The toy model is written by the same code that reads it, so no tensor is ever unmatched. The model loaded successfully and generated fluent nonsense.
Model weights were never moved to the device. HeliosModel(device=...) only records the device for tensors it creates at runtime; the loader called .to(dtype), which is not .to(device)	On a single-device machine the requested and default devices agree, so the mismatch cannot occur. The regression test asserts the invariant — every parameter on the device inputs are built on — not the symptom.
The runner converted the resulting exception into a retryable fault, so a forward pass that always failed became an infinite preempt-and-retry loop whose only symptom was an empty result list	Correct behaviour in production and useless for diagnosis. A smoke test now drives the raw executor with exceptions unhandled before the scheduler is involved.
The engine ran real weights in fp32 on the GPU	Not a correctness bug. Now fp16 on CUDA, decided once and shared by the weights, the KV cache and the block-size arithmetic.

A fifth issue cost a session without being a code bug: Kaggle's working directory persists between sessions, so cloning into an existing directory fails while the notebook continues, and the run silently executes the previous session's code. The reported failures were bugs already fixed upstream. The runner now aborts if the checkout is behind origin.

14 Reproduction

```
# tests
PYTHONPATH=python python -m pytest tests -q

# simulation; any failure replays from its seed
PYTHONPATH=python python -m helios.cli vopr --seeds 50000
PYTHONPATH=python python -m helios.cli vopr --seed 918273 --replay

# ablations -> JSON artifacts -> generated docs/BENCHMARKS.md
python bench/loadgen.py --model artifacts/bench_model --suite all
python bench/loadgen.py --model artifacts/bench_model --suite spec \
  --requests 16 --prompt-len 48 --output-len 24
python bench/report.py

# acceptance rate and its significance analysis
python bench/measure_alpha.py --model artifacts/qwen05b --gamma 4
python bench/measure_alpha.py --model artifacts/qwen05b --no-awq \
  --out artifacts/alpha_rtn.json
python bench/alpha_significance.py

# real checkpoint end to end (downloads Qwen2.5-0.5B if absent)
python bench/real_model_check.py --model artifacts/qwen05b --device cpu

# GPU gates: device report, tests, Triton parity, real model, benchmarks
python bench/kaggle_gpu_run.py

# this report
python bench/make_final_report.py --out docs/helios_report.pdf
```

Field	Value
Commit	06a9055

Generated	28 July 2026
Python / torch	3.12.0 / 2.5.1+cpu
Platform	Windows-11-10.0.26200-SP0
Tests collected	269
Simulation sweep	50,000 seeds, 0 failures
Sweep with the transfer FSM	5,000 seeds, 0 failures
Acceptance measurement	qwen05b, gamma=4, group=128, awq=True

Every number in this document is read from a JSON artifact under `artifacts/` at generation time by `bench/make_final_report.py`. None is typed in.

Artifacts not present at generation time, listed rather than omitted: `gpu_run.json`, `vllm_baseline.json`. The vLLM baseline has never been run; the GPU run record was produced in a Kaggle session and its numbers are quoted from the commit messages and docs that recorded them.